MARKETPULSE

Lakehouse Architecture & Business Intelligence

DS-3001 / Data Analysis and Visualization / Phase 3

Document status
FINAL EVIDENCE PACKAGE

Engineering and report: Soban (24L-2545)
Power BI: Saad Jahangir (24L-2516)
Earlier phases: completed jointly by Soban and Saad.

Purpose

MarketPulse converts daily market prices into a reusable analytical model. The pipeline retains raw inputs, validates and reconciles typed records, and computes business measures before Power BI reads them. Its three questions concern closing-price trends, relative volatility, and trading volume versus price movement.

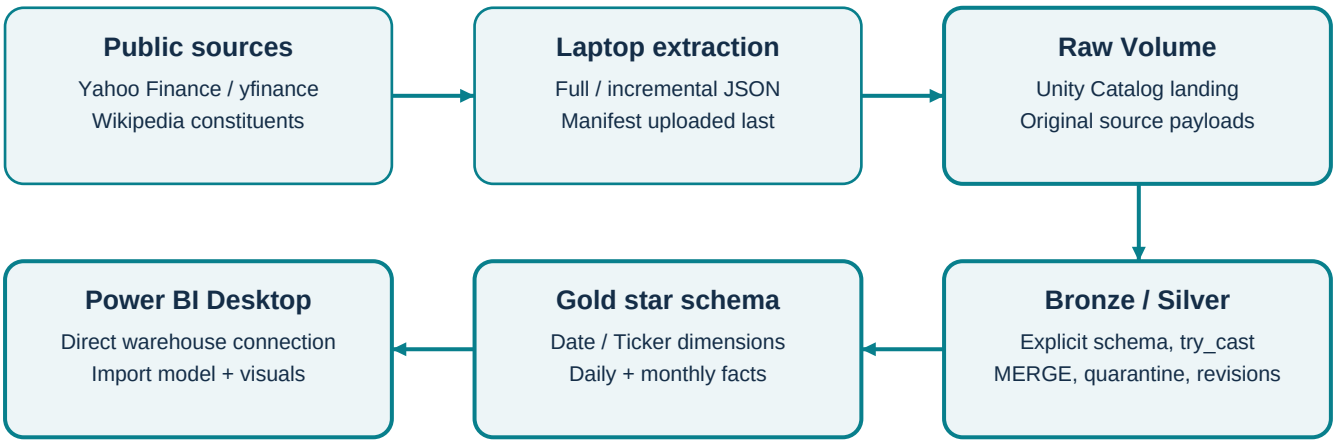
Scope and evidence

The Phase 1 proposal used eight tickers and five years. Phase 2 expanded extraction to the current S&P constituent universe and a ten-year baseline. The saved Phase 2 idempotency export documents 1,225,467 Silver price rows. That historical evidence is separate from validation of the new Gold implementation.

Item	Status
Phase 3 workspace run	364924820876832
Gold integration notebook 82	Confirmed passed
Power BI deliverable	Provided

Repository: github.com/Soban06/MarketPulse

02 / End-to-end architecture



All Spark layers share execution logging; Gold validation gates BI refresh.

Deployment: Databricks Free Edition (AWS), serverless compute + SQL warehouse.

Execution boundary

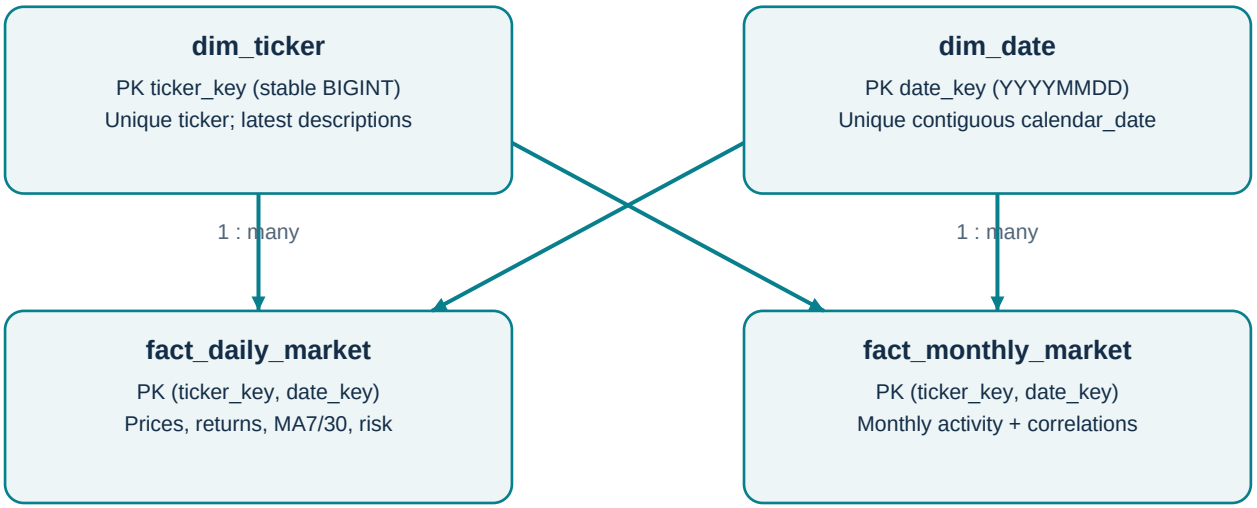
Source extraction runs on the laptop because the implemented Free Edition workspace restricted outbound access to the source hosts. The uploader writes manifest.json last. A batch without a manifest is incomplete and is ignored. Raw payloads live in a Unity Catalog Volume; Bronze/Silver/Gold tables are managed Delta tables.

Pipeline integration

The checked-in Databricks bundle runs setup and then notebook 04. The Python pipeline entry point calls Bronze, Silver prices, Silver profiles and Gold in sequence under a shared pipeline_run_id. It does not continue into Gold after upstream failure. The local runner combines extraction, upload and job invocation into one daily command.

The job permits one concurrent run. Gold always runs after Silver, including a retry where Silver has no new pending batches. Automatic Power BI Service refresh is not assumed: the Desktop file is refreshed only after the entire job succeeds.

03 / Gold dimensional model



Object	Business grain / role
dim_ticker	One company symbol; stable numeric key, latest Type 1 descriptive attributes.
dim_date	One contiguous calendar date, including weekends; no fabricated daily price rows.
fact_daily_market	One ticker per observed trading date; prices and derived business metrics.
fact_monthly_market	One ticker per observed month; aggregate counts, volume and correlations.

Facts reference both dimensions. The monthly date key is the first calendar date of the month. Daily and monthly facts are never joined directly in Power BI. Primary and foreign keys are informational in Databricks; explicit key, cardinality and orphan checks establish integrity. NOT NULL constraints are enforced.

Ticker descriptions use Type 1 updates. Existing keys are retained when new symbols arrive or profiles are absent. Historical sector views therefore use current known classification. A complete column dictionary and ER diagram are versioned with code.

04 / Metric definitions and analytical meaning

Metric	Definition / interpretation
Daily return	Current adjusted close / previous observation adjusted close - 1. First or invalid adjacent prices produce null.
MA 7 / MA 30	Arithmetic mean of close over 7 / 30 trading observations, current included. Full valid windows required.
Rolling volatility	Sample standard deviation over 30 valid daily returns. Earliest value is observation 31.
Annualized volatility	Rolling volatility x sqrt(252); a convention, not a risk forecast.
Relative volume	Current volume / mean of preceding 30 observations. Excludes current row; zero/incomplete baseline produces null.
Monthly correlation	Pairwise Pearson correlation of raw/relative volume with absolute daily return within a ticker-month; undefined variance gives null.

Windows are computed on complete Silver history before Power BI filters. Trading observations bridge weekends and holidays without inserting false zero returns. Prices, moving averages, volatility and correlations are non-additive. Volume and observation counts can be summed. Percentages remain decimal fractions and receive percentage formatting only in Power BI.

Limits on interpretation

Volume association does not establish causation. The current constituent universe creates survivorship bias in historical comparisons. Current company-profile metrics are snapshots, so this model does not claim historical valuations or historical index membership. The monthly summary does not assert every month has complete market coverage.

05 / Engineering decisions and governance

Decision	Reason and consequence
Explicit schemas	Bronze source values are read as strings. Unknown price/constituent fields are rescued; Silver try_cast and quarantine handle invalid types.
Conditional MERGE	Natural keys reconcile Silver. Gold recalculates complete history but writes only changed values, preserving timestamps on identical reruns.
Pinned source versions	Each Gold run reads fixed Silver Delta versions and records them in audit parameters. Run one writer at a time.
Full-history windows	Backfilled observations and price revisions can affect subsequent returns/windows. Full recomputation favors correctness at the present scale.
No Hive partitions	This dataset is far below 1 TB. Many small ticker/day partitions add overhead. Optional clustering targets trade_date and ticker_key.
Operational views	BI reads execution steps and aggregated quarantine/revisions through Gold views. Reject totals are taken from quarantine, not summed repeatedly across step logs.

PII and access

OHLCV records are public company-market observations. However, extraction now stores complete Yahoo profile payloads, which can include public officer names and other personal attributes. Raw must not be described categorically as PII-free without payload review. Explicit downstream schemas select only approved company fields; Gold excludes officer details. Audit identities stay in ops and are omitted from the BI execution view. Review error messages before publication. Credentials and landing batches are git-ignored; screenshots must redact secrets and unnecessary identities.

06 / Validation, logging and recovery

The local analytical tests execute the production SELECT statements in DuckDB with synthetic data and independently calculated expectations. They cover moving averages, adjusted returns, volatility, volume baselines, correlations, invalid values and correction/backfill propagation. This is an independent SQL calculation check, not a substitute for Spark/Delta integration testing.

Local verification: **38 tests passed**. The successful Databricks integration export is saved separately with its real outputs.

Notebook 82 tests real Delta operations in uniquely named scratch schemas: audit-schema migration, allocated keys, missing profiles, identical reruns, historical corrections, source deletions, failed preflight and retry. Existing Phase 2 tests are extended so their cleanup cannot target production Gold.

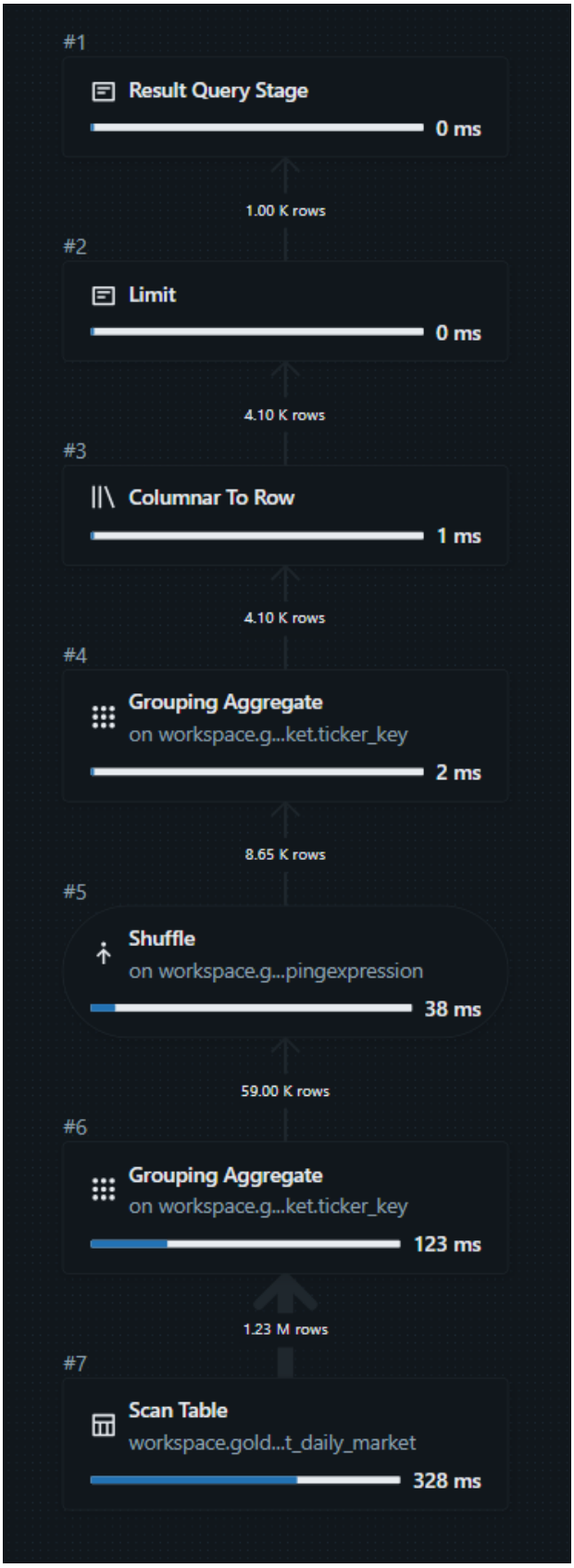
Gold reconciliation	Measured value
Daily fact rows	1230497
Monthly groups	59004
Daily observations represented in monthly groups	1230497
Workspace integration confirmation	Passed

Failure behavior

Gold stages each result once, checks keys, then uses Delta MERGE. Empty Silver prices fail preflight and preserve existing Gold facts. Actual row inserts, updates and deletes come from Delta metrics; rows_aggregated counts monthly output groups. Timings include calculation, validation and writes. A failed final audit write fails the pipeline so success is not silently reported without its log.

Delta commits are atomic per table, not across the whole star schema. A failure can leave partially refreshed Gold tables. A complete successful retry is required before Power BI Import refresh. Staging cleanup errors are surfaced for follow-up.

07 / Executed DAG and performance evidence



Executed graph

Read from bottom to top: Spark scans Gold daily facts, computes partial ticker-month aggregates, shuffles those groups between tasks, and combines them in a later grouping aggregate. Columnar To Row converts the result format before a Limit and Result Query Stage return the notebook preview. The visible Shuffle confirms data redistribution during aggregation. The runtime graph complements the separately captured initial EXPLAIN plan.

Displayed operator timings

Scan Table: 328 ms
Grouping Aggregate (lower): 123 ms
Shuffle: 38 ms
Grouping Aggregate (upper): 2 ms
Columnar To Row: 1 ms
Limit: 0 ms
Result Query Stage: 0 ms

Rows and preview limit

The graph scans about 1.23 million rows. Its Limit operator returns a 1,000-row notebook preview. Intermediate row labels describe this limited execution; they do not replace the independently validated 59,004-row monthly fact count.

Timing interpretation

The matching performance summary shows 1.312 seconds and 21.20 MB read. Operator timings are not added to infer end-to-end duration. Scan Table is the largest displayed operator. This query has no WHERE predicate, so predicate pushdown is not claimed.

Submission note

The instructor accepts this Databricks Query Profile as the Free Edition alternative to Spark UI, as confirmed by Soban.

Separate repeated analytical reads: [1.105, 0.915, 0.941] seconds; caching may affect results. OPTIMIZE was not run and no post-optimization speedup is claimed.

08 / FinOps evaluation

Observed incremental run	Evidence
Run id	364924820876832
Spark pipeline elapsed time	296.817 seconds
Summed logged step time	224.012 seconds
Billed DBUs	Awaiting measurement
Actual run cost	0.000 USD

Measured incremental catch-up batch (11,063 input price rows; 5,030 new observations and 346 revisions) including the first Gold build. This is not a steady-state daily median.

Full-scale unchanged-input refresh: 122.715 seconds; Gold inserts/updates/deletes: 0/0/0. This measures Gold steady-state work with no newly landed batch, not full weekday ingestion.

The implemented platform is Databricks Free Edition on AWS, rather than a measured Azure for Students deployment. A confirmed Free Edition bill can be zero while compute quota is still consumed. No Azure credit usage or billed DBUs are invented. Wall time and summed step durations are operational measurements, not interchangeable with billed compute. Capture a normal daily run separately from full-load and rerun demos.

Enterprise hourly scenario

Hourly 24/7 operation implies **8,760 runs per year**. Annual estimate = 8,760 x enterprise per-run cost + 12 x (monthly storage + BI + other recurring costs). Per-run cost must reflect DBU/SKU price and separately billed VMs when relevant, plus startup, minimum billing, idle capacity and retries. Serverless infrastructure must not be counted twice.

Enterprise assumption	Value
Region	Azure East US - illustrative paid scenario
Estimated cost per run	0.450 USD
Estimated annual architectural cost	5,778.000 USD

Planning scenario, not measured Azure billing: assume 1 billed DBU per complete serverless job at the official East US Premium Automated Serverless Compute rate of USD 0.45/DBU. Allow USD 5/month storage, USD 48/month for two Power BI Premium Per User licenses (24/user/month paid yearly), and a USD 100/month budget for a short-lived serving warehouse, requests and transfer. These storage/serving amounts are budgets, not vendor quotes. Hourly 24/7 = 8,760 runs/year. No separate VM charge is added to serverless. Before procurement, benchmark paid DBUs and warehouse idle time; 0.5/1/2 DBU per run implies annual totals of USD 3,807 / 5,778 / 9,720 under the same monthly budgets. Taxes, growth and source licensing are excluded. Free Edition exposes no billed DBUs here, so wall time is not converted into fabricated usage.

Cost improvements: process after daily source updates; use sampled development data; terminate idle warehouses; reuse materialized stages; tune Import refresh frequency; consider affected-ticker recalculation only if measured full-history cost warrants it.

09 / Power BI and submission readiness

The completed Power BI Desktop report and PBIX are supplied. Soban confirmed the refresh and slicer checks and approved the three final screenshots on 8 November 2026. This confirmation is supplied by the project owner; the report builder does not execute Power BI Desktop.

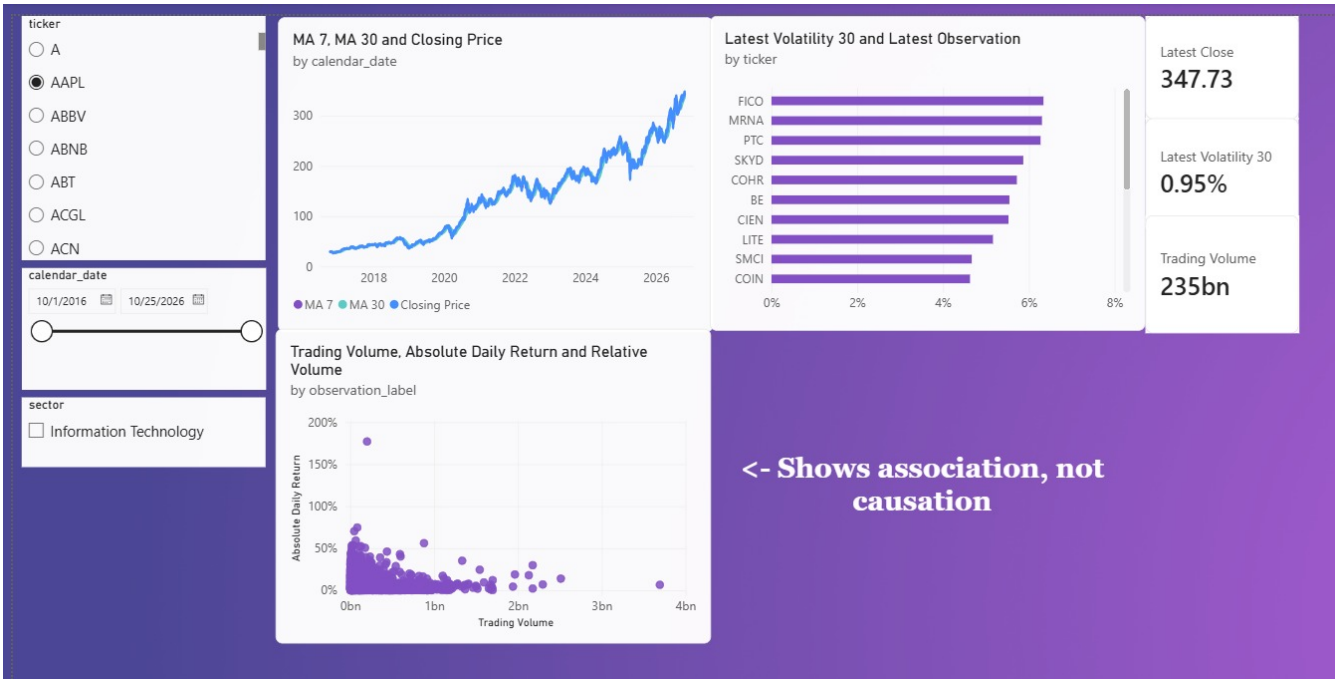


Figure 9. Market analysis: selected-ticker close and MA7/MA30, cross-ticker latest volatility, ticker-date volume/absolute-return scatter, KPI cards and slicers.

Power BI Desktop connects directly to the Databricks SQL warehouse using Import mode. A single-ticker price chart avoids summing company prices. Risk comparisons use each ticker's latest available 30-return volatility within the selected dates; tooltips expose stale observation dates. Scatter points retain ticker-date grain.

Operations show execution status, duration, actual quarantine reasons and revision counts. Summing rejected-row counters across multiple logged steps would double-count rejects; the quarantine view supplies those totals instead.

The full Phase 3 submission consists of the updated GitHub repository, a Power BI deliverable and the architecture PDF. The checked-in handoff guide lists the exact connections, measures and remaining screenshots.

10 / Power BI pipeline health

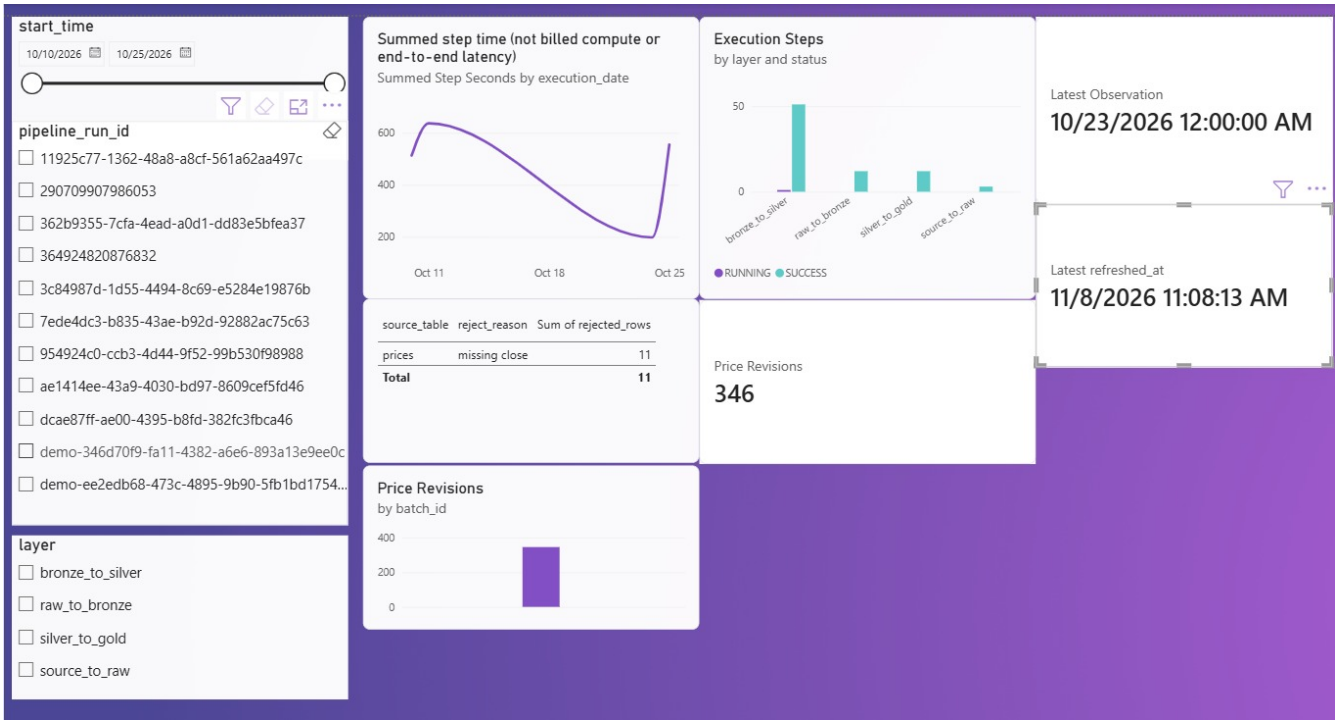


Figure 10. Operational dashboard with execution statuses, summed step duration, rejected-row reasons, revision counts and refresh information.

The displayed latest market observation is 23 October 2026; the displayed import refresh is 8 November 2026 at 11:08:13. A successful Import refresh reloads the available Gold data; it does not itself extract new market observations. The screenshot includes historical and demo run identifiers, so its status counts describe the selected log history rather than only the latest production run.

11 / Power BI model relationships

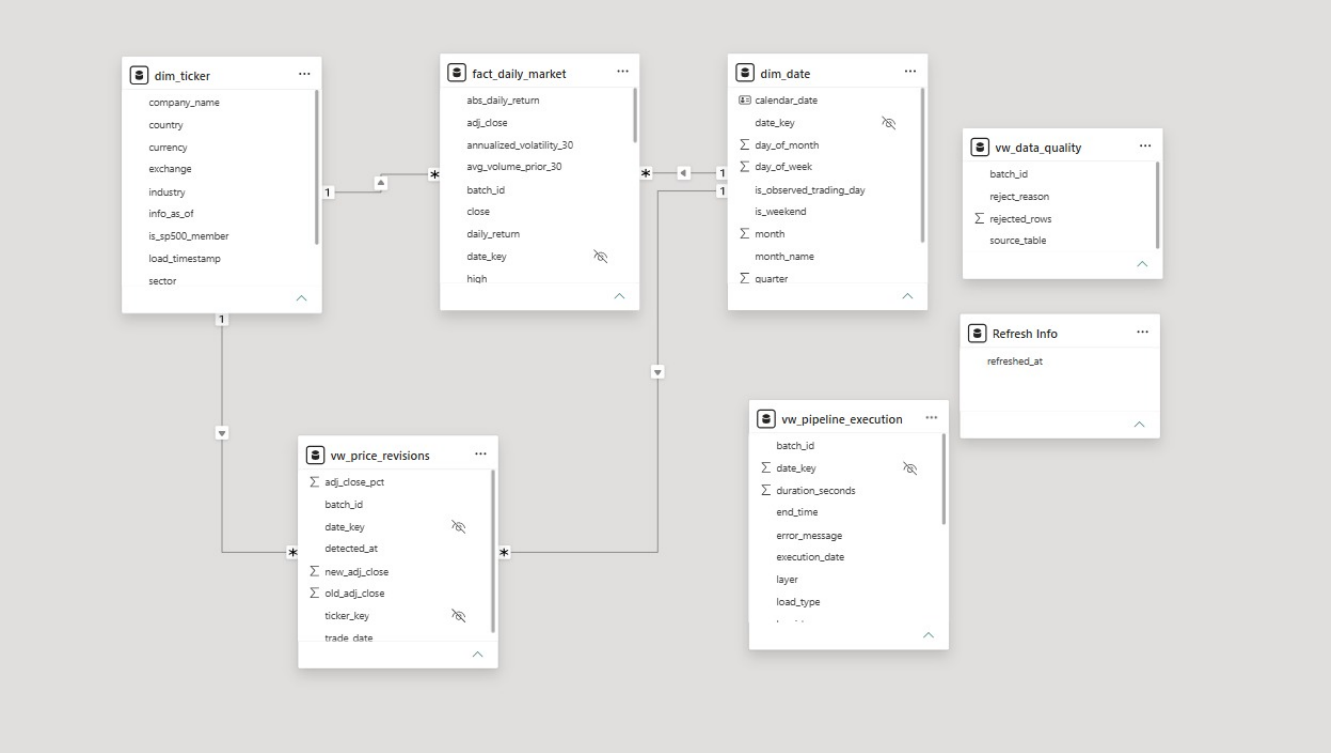


Figure 11. Actual Power BI Model view: Date and Ticker dimensions filter daily market facts and price revisions through one-to-many relationships. Execution, data-quality and refresh-information tables remain separate.

Deliverable: bi/MarketPulse.pbix, stored using Git LFS. The model screenshot documents the Desktop relationship layout; the Gold star schema alone was not used as a substitute for this evidence.

12 / References and evidence register

Project source and version history
https://github.com/Soban06/MarketPulse

Databricks constraint semantics
https://docs.databricks.com/aws/en/tables/constraints

Databricks partitioning guidance
https://docs.databricks.com/aws/en/tables/partitions

Serverless limitations / Spark UI
https://docs.databricks.com/aws/en/compute/serverless/limitations

Query Profile
https://docs.databricks.com/aws/en/sql/user/queries/query-profile

Power BI Databricks connector
https://learn.microsoft.com/en-us/power-query/connectors/databricks

Serverless bundle configuration
https://docs.databricks.com/aws/en/dev-tools/bundles/examples

Scenario pricing source: https://prices.azure.com/api/retail/prices ;
https://www.microsoft.com/en-us/power-platform/products/power-bi/pricing (checked 2026-10-25; details in docs/phase3/FINOPS.md)

Repository evidence

Evidence	Location / meaning
Original business questions	Phase 1 proposal supplied by Soban; three questions retained.
Phase 2 saved runs	docs/run-evidence/80, 81, 90 and 91 HTML exports; historical evidence.
Gold integration	notebooks/82_test_gold.py; export actual outputs after workspace pass.
DAG / runtime / layout	notebooks/93_phase3_evidence.py; actual measurements and screenshots.
Metric and BI contract	docs/phase3/GOLD_MODEL.md, POWER_BI_HANDOFF.md, bi/measures.dax.
Report input evidence	docs/report/evidence.json; no invented results permitted.

All automated evidence-presence checks passed. Review the real screenshots, assumptions and deliverables before submission.